

Universidad de Antioquia

Ingreniería de telecomunicaciones
Informatica 2

Sweet Crush

Proyecto de programación

Estudiante:

Andrés Felipe Sierra Fernández
David Guardiola

Docente:

Anibal Jose Guerra

September 14, 2026

1 Contextualización

El desafío trata sobre un juego de tipo *arcade* conformado por diferentes fichas, en el cual el jugador podrá eliminar fichas del tablero, una por ronda, modificar la estructura del tablero, detectar combinaciones, entre otras acciones.

El juego exige que el tablero sea representado de una manera diferente a una matriz convencional de valores enteros. Las fichas deben estar representadas mediante una secuencia de bits empaquetados en bytes, haciendo que la forma de almacenar y manejar el tablero sea una parte importante del desafío.

Además, durante el juego el tablero puede cambiar su estructura, permitiendo agregar o eliminar filas y columnas, y se deben procesar las combinaciones que se produzcan después de las diferentes acciones realizadas. “

2 Análisis

El juego requiere que cada posición del tablero sea representada mediante 3 bits. Debido a que con 3 bits es posible representar 8 combinaciones diferentes, se utilizarán 6 de estas para representar las distintas fichas, una para representar una posición vacía y la última para representar una combinación especial.

Para implementar esta representación del tablero, es necesario desarrollar funciones que permitan administrar la memoria y realizar la manipulación de los bits correspondientes a cada ficha. Para esta primera parte del proyecto se utilizarán las siguientes funciones:

- `int calcularBytesTablero(int filas, int columnas)`: se encarga de calcular la cantidad de bytes necesarios para almacenar el tablero de acuerdo con sus dimensiones, teniendo en cuenta que cada posición requiere 3 bits.
- `unsigned char* crearTablero(int filas, int columnas, int bytes)`: se encarga de crear el tablero utilizando memoria dinámica, a partir de sus dimensiones y de la cantidad de bytes calculada.
- `void inicializarBuffer(unsigned char* buffer, int bytesTablero)`: recibe un puntero a la memoria del tablero y la cantidad de bytes que este utiliza, para inicializar la memoria con ceros. Estos valores serán modificados posteriormente a medida que se almacenen las fichas.

- `int calcularPosicion(int fila, int columna, int columnas)`: calcula la posición lineal correspondiente a una determinada fila y columna del tablero, permitiendo posteriormente acceder a la ficha almacenada en dicha posición.
- `void guardarFicha(unsigned char* memoria, int posicion, unsigned char ficha)`: recibe un puntero a la memoria del tablero, la posición de la ficha y el valor correspondiente a esta. Su función es almacenar los 3 bits de la ficha en la posición correspondiente dentro de la secuencia de bits.
- `unsigned char obtenerFicha(unsigned char* memoria, int posicion)`: se encarga de extraer los 3 bits correspondientes a una ficha a partir de su posición dentro de la memoria, permitiendo recuperar el valor almacenado.

Estas funciones constituyen la base para la implementación del tablero y permiten abstraer las operaciones relacionadas con la administración de la memoria y la manipulación de los bits. A medida que avance el desarrollo del programa, se implementarán nuevas funciones encargadas de las demás operaciones requeridas por el juego, como la detección de combinaciones, eliminación de fichas y modificación de la estructura del tablero. Una vez establecida esta base, se podrá continuar con la implementación de la parte visual del juego. “ “ “ “ latex

3 Diseño

Para la representación del tablero se decidió almacenar las fichas utilizando grupos de 3 bits dentro de una secuencia de bytes. Los bits correspondientes a cada ficha se van almacenando de izquierda a derecha dentro de cada byte, teniendo en cuenta que una ficha puede atravesar el límite entre dos bytes consecutivos.

Un ejemplo de la representación del tablero en memoria es el siguiente:

```
Byte[0]: 00101001
Byte[1]: 11001011
Byte[2]: 01100011
Byte[3]: 01000100
Byte[4]: 00010001
Byte[5]: 00101010
Byte[6]: 01110010
Byte[7]: 10100000
```

Bytes del tablero: 8

A partir de esta representación, la función `obtenerFicha` permite extraer el valor correspondiente a cada posición del tablero.

Posición	Valor de ficha	Representación binaria
0	1	001
1	2	010
2	3	011
3	4	100
4	5	101
5	5	101
6	4	100
7	3	011

Para realizar esta operación se debe tener en cuenta la posición de los 3 bits dentro de la memoria, ya que algunas fichas pueden encontrarse completamente dentro de un byte, mientras que otras pueden estar divididas entre dos bytes consecutivos. En este último caso, la ficha puede tener dos bits almacenados en un byte y el bit restante en el siguiente, o un bit en un byte y los otros dos en el siguiente.

Una vez obtenidos los valores de las fichas, se utiliza un ciclo para recorrer las posiciones del tablero y representarlo en consola. De esta manera, la información almacenada linealmente en memoria puede ser mostrada al usuario como filas y columnas:

```
fila 0: B C D E
fila 1: F F E D
fila 2: C B A B
fila 3: A E F C
fila 4: D E F C
```

En el estado actual del desarrollo, el programa permite calcular la cantidad de bytes necesarios para el tablero, crear la memoria dinámica correspondiente, calcular la posición lineal de cada ficha, almacenar y obtener sus valores y finalmente mostrar el tablero en pantalla.

El siguiente es un ejemplo de ejecución del programa:

La siguiente etapa del desarrollo consiste en implementar la redimensión del tablero. Para esto será necesario calcular nuevamente la cantidad de bytes requerida, crear un nuevo bloque de memoria, reorganizar las fichas de acuerdo con las nuevas dimensiones, liberar la memoria utilizada por el tablero anterior y finalmente mostrar la nueva estructura del tablero. “

“`latex`

3.1 Generación y llenado del tablero

Una vez implementadas las funciones necesarias para almacenar y recuperar fichas mediante la representación de tres bits, se desarrollaron las funciones encargadas de generar las fichas y llenar el tablero.

La función `generarFicha()` se encarga de generar aleatoriamente el tipo de ficha que será almacenado. Debido a que el juego cuenta con seis tipos diferentes

de fichas, se utiliza la función `rand()` junto con el operador módulo para obtener un valor en el rango de 0 a 5.

```
unsigned char generarFicha(){  
    return rand() % 6;  
}
```

Por otra parte, la función `llenarTablero()` recorre todas las posiciones del tablero mediante las filas y columnas. Para cada posición recorrida se genera una ficha aleatoria y posteriormente se calcula su posición lineal dentro de la secuencia de fichas. Finalmente, la ficha generada es almacenada en dicha posición mediante la función `guardarFicha()`.

```
void llenarTablero(unsigned char *tablero, int filas, int columnas){  
    for(int fila = 0; fila < filas; fila++){  
        for(int columna = 0; columna < columnas; columna++){  
            unsigned char ficha = generarFicha();  
            int posicion = calcularPosicion(fila, columna, columnas);  
            guardarFicha(tablero, posicion, ficha);  
        }  
    }  
}
```

De esta manera, las posiciones del tablero son recorridas de forma ordenada, mientras que el valor de la ficha asignada a cada posición se genera aleatoriamente. La función no modifica el orden de las posiciones ni realiza una distribución aleatoria de estas, sino que genera un valor diferente para cada posición durante el recorrido. ““

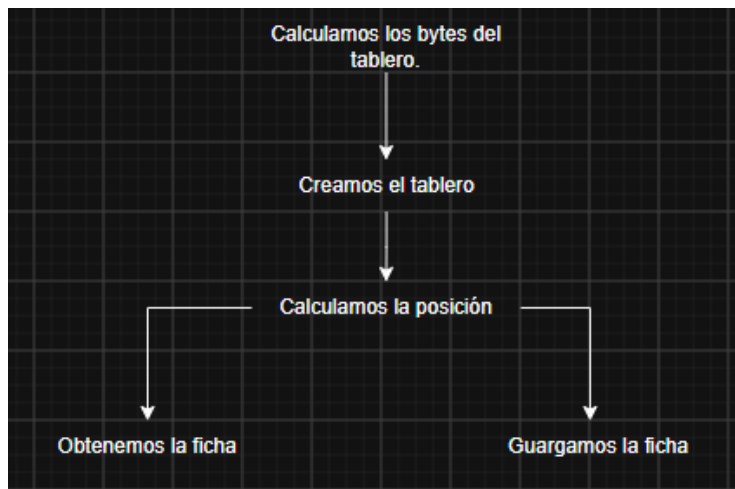


Figure 1: Ejecución del programa mostrando la representación del tablero.